

CSCE 638: Human Chess Move Prediction for Predictive Caching

Hung Nguyen
737002625

nguye3hv@tamu.edu

Mukil Senthilkumar
836000195

mukilsenthil@tamu.edu

Seyed Ali Ghazi Asgar
532007084

alighazi@tamu.edu

Arif Nizami
137009847

arifnizami003@tamu.edu

1 Introduction

Chess has long served as a benchmark for artificial intelligence, with modern systems combining powerful search and learned evaluation (Silver et al., 2017; The Stockfish Developers, 2024). Recent studies have also explored using language models to represent chess states and generate moves from textual encodings such as PGN or FEN (Feng et al., 2023; Kolasani et al., 2025). These developments motivate exploring whether LLMs can be used for chess move prediction.

In this project, we propose a chess move prediction system that anticipates likely human moves and precomputes candidate responses. Given the current board state, move history, and player-skill information such as Elo rating, the system predicts the opponent’s top- k next moves and uses a chess engine to compute responses in advance. If the actual move matches one of these predictions, the system can return a cached response immediately. Our goal is to test whether a LLM can better predict human next moves than Stockfish-based baselines, and whether these improvements lead to higher cache hit rates and lower effective response latency through predictive caching. We formulate this as a sequence modeling task over structured chess representations such as board state and move history.

2 Related Work

Chess AI has progressed from classical search-based systems to neural and language-model-based approaches. Recent work has explored language models for representing chess games as sequences and generating moves from textual encodings such as Portable Game Notation (PGN) or Forsyth-Edwards Notation (FEN) (Feng et al., 2023). At the same time, modern engine systems such as AlphaZero-style models and Stockfish have shown that high-level play can be achieved through powerful search and learned evaluation (Silver et al., 2017; The Stockfish Developers, 2024).

A related line of work studies human-centered chess modeling rather than pure engine strength. Models like Maia-2 (Tang et al., 2024) and Deep Factorization Machines (Rokach and Shapira,

2026) use personalized embeddings to capture specific player strengths and "blunder profiles." In gaming networks, deep learning effectively predicts player inputs up to 50ms ahead to mask perceived latency (Halbhuber et al., 2022; Motoo et al., 2021; Gutmann and Konagaya, 2019).

Outside of chess, predictive computation has been used to reduce perceived delay in interactive systems and networked environments by estimating future actions and preparing responses in advance (Motoo et al., 2021; Gutmann and Konagaya, 2019; Halbhuber et al., 2022). Prior work has studied human move prediction in chess (Tang et al., 2024; Rokach and Shapira, 2026) and predictive techniques for reducing perceived latency in interactive systems (Halbhuber et al., 2022; Motoo et al., 2021; Gutmann and Konagaya, 2019), but not in a setting that uses predicted human chess moves to drive pre-computed engine responses. Our project connects these areas by combining human move forecasting with predictive caching.

3 Methodology

3.1 Problem Definition

Let s_t denote the current board state at time t , e denote player-skill information such as Elo rating, and $H = \{m_1, m_2, \dots, m_{t-1}\}$ denote the sequence of previous moves. Given s_t , H , and e , our task is to predict a ranked set of likely next moves:

$$M_k = \{\hat{m}_1, \hat{m}_2, \dots, \hat{m}_k\}$$

We then use a chess engine to precompute a response for each move in M_k . If the true opponent’s move $m_t \in M_k$, the system can immediately use the cached response. Otherwise, it falls back to standard on-demand engine computation. Our research question is whether LLM-based human move prediction can better match actual human next moves than Stockfish-based baselines, and whether those gains lead to higher cache hit rates and lower effective response delay.

3.2 Proposed Approach

We represent the current position using Forsyth–Edwards Notation (FEN) and the move history using Portable Game Notation (PGN). These representations allow us to convert each game state into a structured text sequence suitable for model input.

We will adapt a pretrained language model using parameter-efficient fine-tuning to predict the next move given the current FEN, recent PGN history, and player-skill information such as Elo rating. This allows the model to learn different move tendencies across novice, intermediate, and advanced players.

The model outputs a ranked top- k list of predicted moves in a standardized chess format, such as SAN. A chess library verifies legality and normalizes outputs, and Stockfish precomputes a response for each legal prediction. If the opponent plays one of the predicted moves, the cached response is used; otherwise, the engine computes a new response.

4 Expected Experiments

To evaluate our approach, we will conduct experiments on next-move prediction accuracy, cache effectiveness, and performance differences across player skill levels.

4.1 Dataset

For this project, we will use the Lichess Rated Standard Chess Games Dataset¹, which contains player ratings, time controls, opening metadata, results, and move sequences. By replaying each game, we create position-level examples with board state, recent move history, and player metadata as input, and the human next move as the label. We will remove invalid games, group players into low, intermediate, and high Elo buckets, and split games into train, validation, and test sets with an 80/10/10 ratio to avoid leakage.

4.2 Baselines

We will compare our proposed system against the following baselines:

- **No-Cache Baseline:** compute the response only after the opponent’s move is observed, without any predictive precomputation.

¹<https://huggingface.co/datasets/Lichess/standard-chess-games>

- **Engine Top-1 Baseline:** predict the opponent’s next move based solely on the highest-ranked engine move.

- **Engine Top- k Baseline:** predict the opponent’s next move using the engine’s top- k recommended moves.

4.3 Planned Experiments

- **Experiment 1: Cache Effectiveness.** We will measure Cache Hit Rate (CHR), defined as the fraction of positions where the true move m_t appears in M_k , and estimate effective response latency by treating cache hits as near-instant retrieval and cache misses as standard engine computation. We will analyze how these metrics change as k varies, while also measuring the extra engine computation required for top- k precomputation.
- **Experiment 2: Comparison with Stockfish Top- k .** We will compare our model and Stockfish using Top- k hit rate. We will evaluate whether our prediction model better matches actual human next moves than Stockfish’s top- k suggestions.
- **Experiment 3: Elo-Level Performance.** We will measure CHR across three Elo buckets to determine whether prediction is easier for lower-rated players, whose moves may be more repetitive or contain more predictable mistakes, than for stronger players.

4.4 Expected Results

We hypothesize that the LLM can outperform Stockfish-based baselines on human move prediction, especially in Top- k hit rate, by capturing plausible human moves that differ from engine-preferred lines. We also expect small values of k , such as 3 to 5, to provide a useful trade-off between cache hit rate and background computation cost. Overall, predictive caching is expected to be most beneficial when the model captures human-likely moves that a standard engine would not prioritize.

5 Expected Timeline

Task	Deadline
Finalize dataset, baselines, and engine setup	March 15
Train prediction model	April 5
Perform experiments	April 12
Prepare presentation and demo	April 15
Write final report	April 26

References

- Xidong Feng, Yicheng Luo, Ziyang Wang, Hongrui Tang, Mengyue Yang, Kun Shao, David Mguni, Yali Du, and Jun Wang. 2023. [Chessgpt: Bridging policy learning and language modeling](#). *Preprint*, arXiv:2306.09200.
- Gregory Gutmann and Akihiko Konagaya. 2019. [Predictive simulation: Using regression and artificial neural networks to negate latency in networked interactive virtual reality](#). *Preprint*, arXiv:1910.04703.
- David Halbhuber, Maximilian Seewald, Fabian Schiller, Mathias Götz, Jakob Fehle, and Niels Henze. 2022. [Using artificial neural networks to compensate negative effects of latency in commercial real-time strategy games](#). In *Proceedings of Mensch Und Computer 2022*, MuC '22, page 182–191, New York, NY, USA. Association for Computing Machinery.
- Sai Kolasani, Maxim Saplin, Nicholas Crispino, Kyle Montgomery, Jared Quincy Davis, Matei Zaharia, Chi Wang, and Chenguang Wang. 2025. [Llm chess: Benchmarking reasoning and instruction-following in llms through chess](#). *Preprint*, arXiv:2512.01992.
- Takato Motoo, Jiei Kawasaki, Takuya Fujihashi, Shunsuke Saruwatari, and Takashi Watanabe. 2021. [Client-side network delay compensation for online shooting games](#). *IEEE Access*, 9:125678–125690.
- Yarden Rokach and Bracha Shapira. 2026. [Blunder prediction in chess](#). *Applied Intelligence*, 56(4).
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2017. [Mastering chess and shogi by self-play with a general reinforcement learning algorithm](#). *Preprint*, arXiv:1712.01815.
- Zhenwei Tang, Difan Jiao, Reid McIlroy-Young, Jon Kleinberg, Siddhartha Sen, and Ashton Anderson. 2024. [Maia-2: A unified model for human-ai alignment in chess](#). *Preprint*, arXiv:2409.20553.
- The Stockfish Developers. 2024. Stockfish: A free and strong open-source UCI chess engine. <https://stockfishchess.org/>. Version 16.1.